TECNOLOGÍAS DE LA INFORMACIÓN
Desarrollo de Sistemas Informáticos
4 créditos

Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: JHONATAN EFRAIN VILLAO TORRES

PARALELO: A

SEMESTRE: QUINTO

ACTIVIDAD: Actividad #1: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

LINK DIAGRAMA: https://drive.google.com/file/d/1qhK3QSwGHYthUWWE4KthUAo9WQCgfG_M/view?usp=sharing

LINK GIT: https://github.com/jvillao4111/S2DSI/tree/main/TEST

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad 1

📘 Actividad #1: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

Unidad: 1. Desarrollo de software Introducción y Paradigmas de diseño

En esta actividad usted deberá diseñar la arquitectura lógica de un sistema de control de rescates y adopciones, integrando estándares de modelado UML, aplicación de patrones de diseño y el uso de herramientas de control de versiones (GitHub/Gitflow).

🎯 Objetivo

Diseñar la arquitectura de un sistema de software aplicando patrones de diseño y flujos de trabajo profesionales, fortaleciendo la capacidad de análisis y las buenas prácticas de ingeniería de software para soluciones escalables y mantenibles.

🧭 Instrucciones

- Realice el análisis del escenario propuesto (identificación de actores y requerimientos).
- Modele el sistema mediante diagramas de casos de uso y clases bajo estándares UML.
- Implemente al menos tres patrones de diseño (Creacionales, Estructurales o de Comportamiento).
- Utilice el flujo de trabajo Gitflow para el registro de avances del diseño en un repositorio de GitHub.
- IMPORTANTE: Revise el detalle del caso de Control de Rescate y la rúbrica en la guía técnica ubicada en el apartado de Recursos de la Unidad 1.

📄 Entrega

Formato: PDF (Debe incluir obligatoriamente el enlace al repositorio de GitHub)
Nombre sugerido: apellido_nombre_actividad1.pdf

SECCIÓN 1: INTRODUCCIÓN Y OBJETIVOS

1.1. Introducción

El presente informe técnico detalla el diseño de la arquitectura lógica para el Sistema de Control de Rescate y Adopción Animal de la Universidad Técnica de Manabí. La gestión operativa de un refugio de fauna requiere un estricto control sobre los flujos de ingreso, estados de tratamiento clínico y procesos de adopción. Para dar solución a esta problemática, se plantea una arquitectura de software modular orientada a objetos que integra patrones de diseño de software (Creacionales y de Comportamiento) bajo principios SOLID, garantizando que el sistema sea escalable y mantenible en el tiempo. Asimismo, el ciclo de vida del diseño se encuentra respaldado bajo la metodología de desarrollo y control de versiones profesional Gitflow.

1.2. Objetivo

Diseñar la arquitectura lógica y el modelo estructural de un sistema informático para la gestión de rescates y adopciones animales, aplicando patrones de diseño de software e integrando flujos de trabajo profesionales en GitHub para asegurar una solución robusta, escalable y con alta trazabilidad.

SECCIÓN 2: ANÁLISIS DEL PROBLEMA (ACTORES Y REQUERIMIENTOS)

2.1. Identificación de Actores Principales

- **Ciudadano / Reportante:** Actor externo encargado de registrar alertas tempranas y notificar de forma detallada el ingreso de animales en condiciones de vulnerabilidad o riesgo callejero.
- **Voluntario / Veterinario:** Personal operativo interno del refugio responsable de la evaluación médica, ejecución de tratamientos clínicos y la actualización del estado de salud del animal.
- **Administrador del Refugio:** Actor con privilegios globales dentro del sistema, encargado de la gestión general del refugio, asignación de personal operativo a los casos y validación de las solicitudes de adopción.

2.2. Requerimientos Funcionales (RF)

- **RF1 - Crear caso de rescate:** El sistema debe permitir al Ciudadano registrar un nuevo caso de rescate, capturando la ubicación geográfica, imágenes y descripción del estado inicial del animal.
- **RF2 - Categorizar especie:** El sistema debe clasificar de manera obligatoria el expediente de ingreso según su taxonomía operativa en: **Perro, Gato o Fauna Silvestre**.

- **RF3 - Asignar voluntario/veterinario:** El sistema debe permitir al Administrador delegar la responsabilidad de un caso específico a un miembro del personal operativo disponible.
- **RF4 - Gestión de estados del proceso:** El sistema debe controlar el ciclo de vida del caso de rescate mediante una transición estricta de estados: **Rescatado ➔ En Tratamiento Redireccionado ➔ Disponible para Adopción**.

2.3. Requerimientos No Funcionales (RNF)

- **RNF1 - Escalabilidad:** La arquitectura lógica debe permitir la adición de nuevas especies (ej. Aves) o nuevos flujos de estado operativo sin alterar el núcleo del código fuente.
- **RNF2 - Mantenibilidad (SOLID):** El diseño de clases debe respetar el desacoplamiento de componentes mediante interfaces claras para facilitar el mantenimiento preventivo y correctivo.
- **RNF3 - Trazabilidad de versiones:** Toda la evolución del diseño de la arquitectura debe registrarse de manera cronológica e histórica utilizando el flujo de trabajo Gitflow en GitHub.

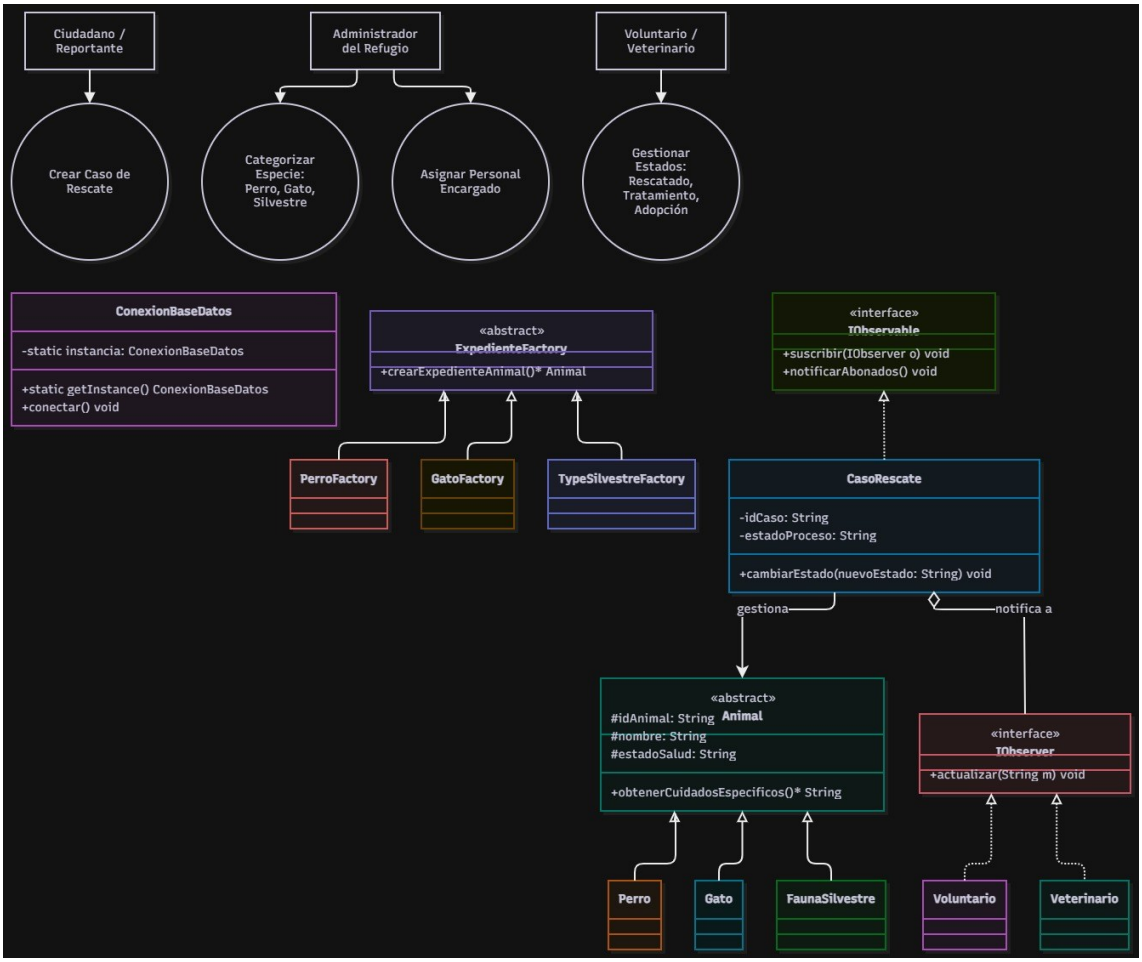
SECCIÓN 3: MODELADO DEL SISTEMA (UML)

3.1. Diagrama de Casos de Uso

Descripción del Diagrama: El diagrama modela el comportamiento del sistema a nivel funcional. Muestra cómo el Ciudadano interactúa de forma directa con el caso de uso de reportes, el Administrador orquesta la asignación y la clasificación taxonómica, y el Veterinario/Voluntario manipula los estados clínicos de la plataforma.

3.2. Diagrama de Clases UML

Descripción del Diagrama: El diagrama expone la estructura interna de la solución informática. Se observa la segregación de responsabilidades bajo principios SOLID. La estructura cuenta con una clase abstracta Animal que utiliza polimorfismo para especializarse en las subclases concretas de las especies correspondientes.



SECCIÓN 4: APLICACIÓN Y JUSTIFICACIÓN DE PATRONES DE DISEÑO

4.1. Patrón Factory Method (Creacional)

- **Problema resuelto:** El refugio gestiona ingresos de múltiples especies (Perros, Gatos, Fauna Silvestre), donde cada animal posee requerimientos, cuidados y fichas de tratamiento radicalmente diferentes. Instanciar estos objetos directamente mediante condicionales if/else en un controlador central acoplaría rígidamente la lógica del negocio.
- **Justificación Técnica:** Se implementa una fábrica abstracta (ExpedienteFactory) con extensiones concretas (PerroFactory, GatoFactory, FaunaSilvestreFactory). Esto aísla por completo la creación de los objetos de su uso operativo. Si el día de mañana el refugio decide atender una nueva especie, solo se creará su fábrica correspondiente sin modificar el código preexistente, cumpliendo con el Principio de Abierto/Cerrado (OCP).

4.2. Patrón Observer (De Comportamiento)

- **Problema resuelto:** Cuando un ciudadano genera un reporte o un animal cambia a estado disponible para adopción, se requiere alertar de forma inmediata a los veterinarios y voluntarios sin acoplar los componentes de comunicación.

- **Justificación Técnica:** La clase CasoRescate actúa como el elemento *Observable* (Sujeto), controlando las suscripciones. Las clases de los usuarios internos (Voluntario, Veterinario) implementan la interfaz IObserver. Ante cualquier cambio en el estado del proceso, se gatilla el método reactivo notificarAbonados(), despachando alertas instantáneas en tiempo real de manera totalmente desacoplada.

4.3. Patrón Singleton (Creacional)

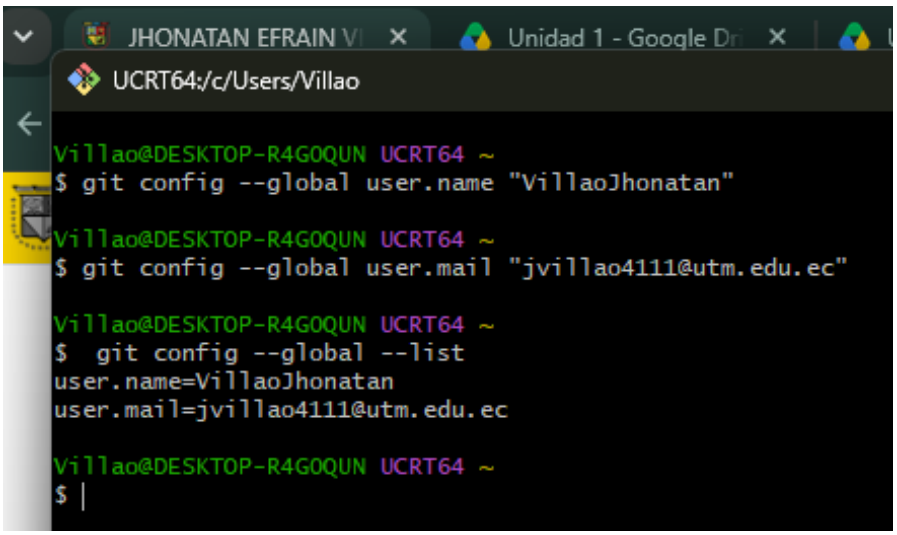
- **Problema resuelto:** Múltiples solicitudes de usuarios concurrentes al motor central de gestión de casos y conexiones de datos pueden saturar la memoria del servidor o causar conflictos de sincronización.
- **Justificación Técnica:** Garantiza que la clase responsable del canal de datos (ConexionBaseDatos) posea **una única instancia global** controlada dentro de toda la aplicación. Se define un constructor privado y se expone su acceso por medio de un método estático público getInstance(), optimizando los recursos del servidor.

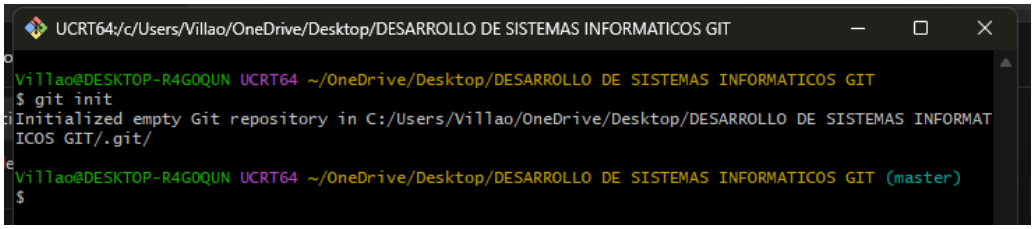
SECCIÓN 5: DOCUMENTACIÓN DEL FLUJO DE TRABAJO (GITFLOW)

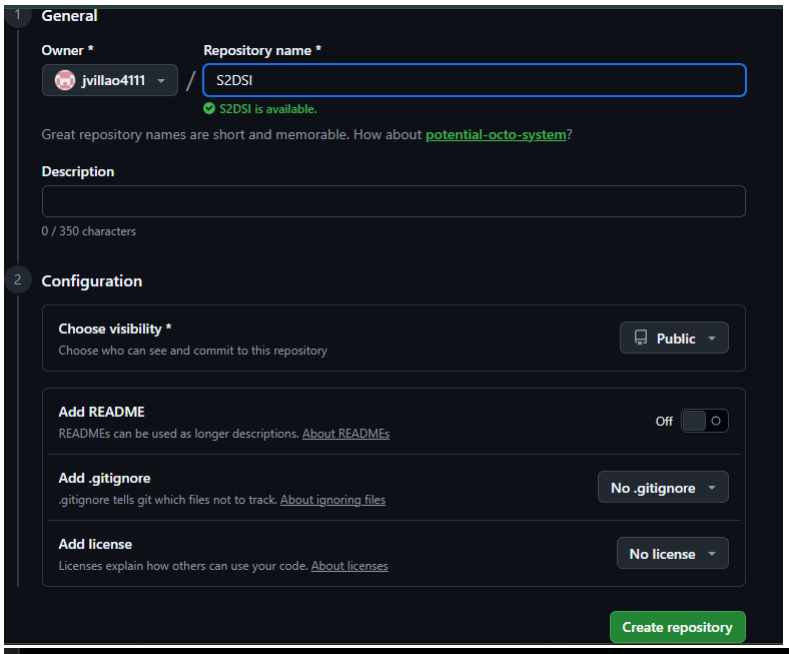
5.1. Justificación de Gitflow

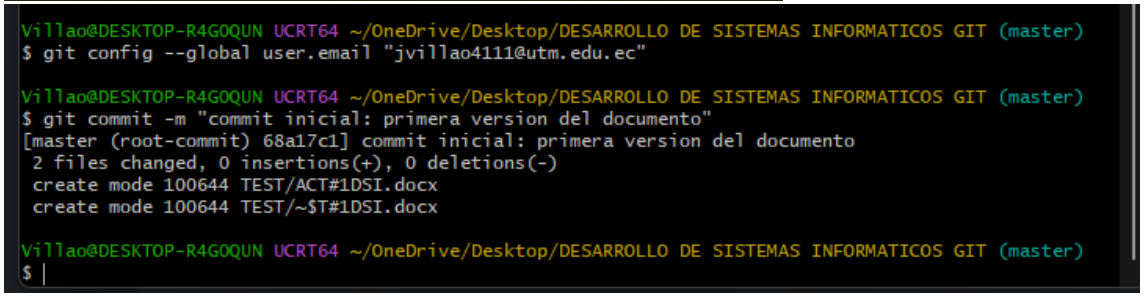
El uso de Gitflow en este proyecto asegura una metodología de trabajo limpia y estándar dentro de la ingeniería de software. Al segmentar las responsabilidades en ramas independientes (main e integración develop), se previene la introducción de errores directos en la versión final de producción y permite el desarrollo simultáneo de características funcionales a través de ramas de soporte (features).

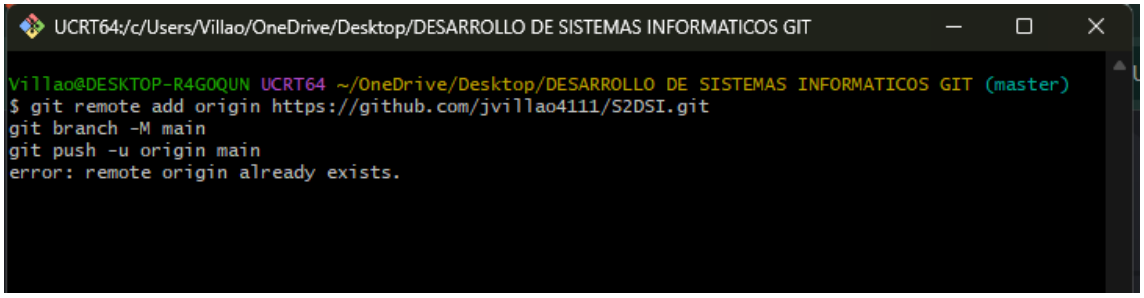
5.2. Evidencias del Flujo de Trabajo

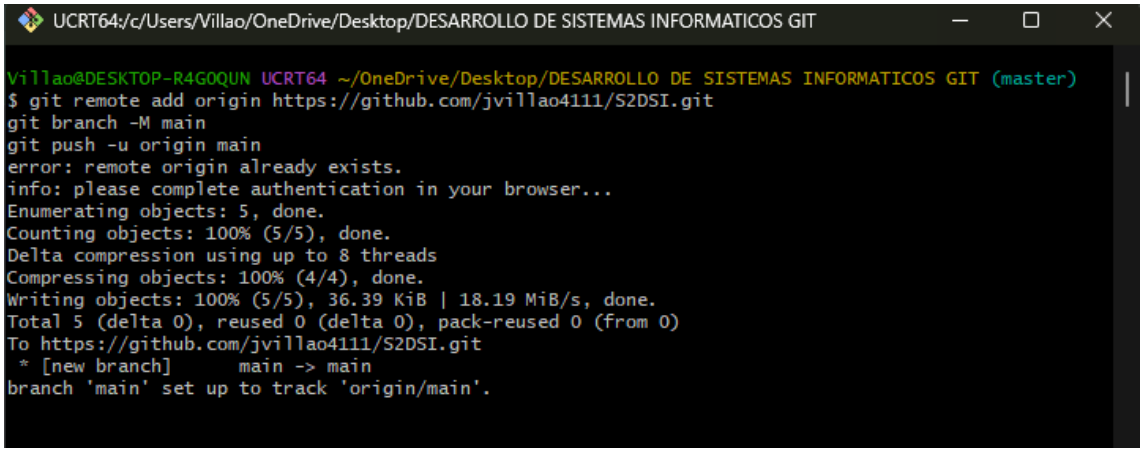


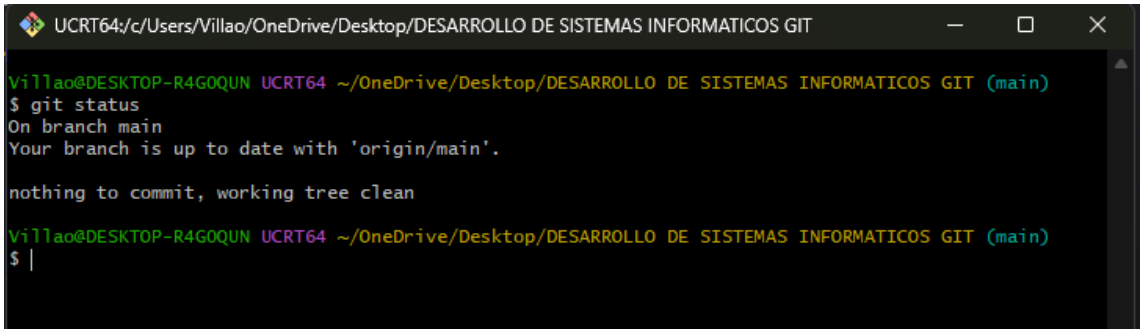

UCRT64:/c/Users/Villao/OneDrive/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS GIT
Villao@DESKTOP-R4GOQUN UCRT64 ~/OneDrive/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS GIT
$ git init
Initialized empty Git repository in C:/Users/Villao/OneDrive/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS GIT/.git/
Villao@DESKTOP-R4GOQUN UCRT64 ~/OneDrive/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS GIT (master)
$

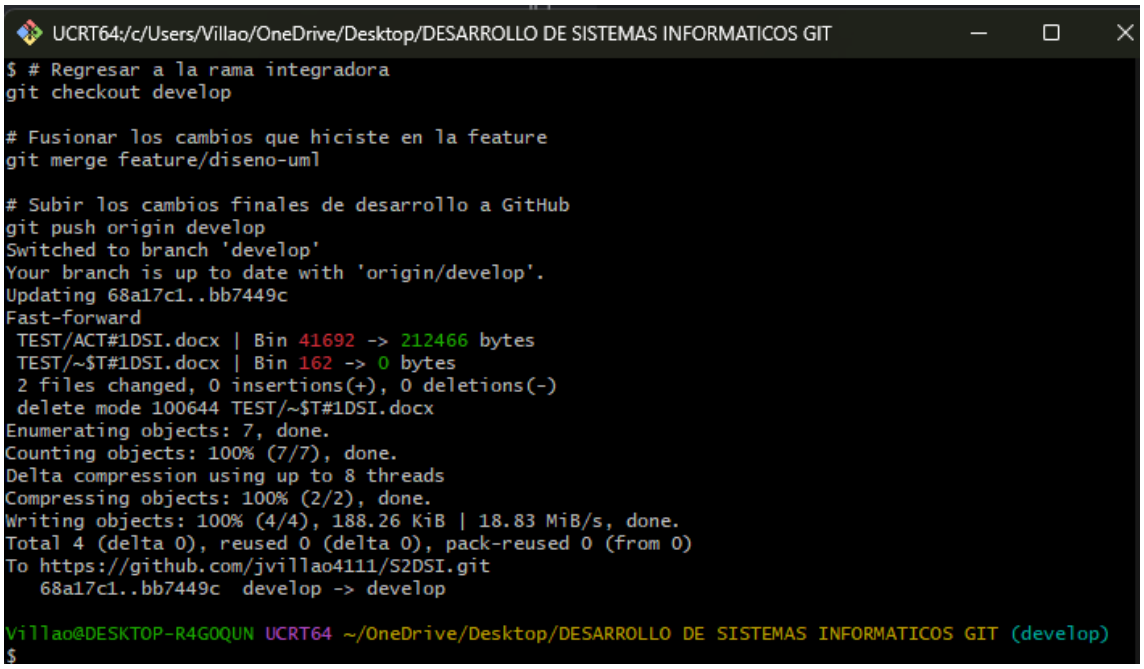

1 General
Owner *
Repository name *
jvillao4111
S2DSI
S2DSI is available.
Great repository names are short and memorable. How about potential-octo-system?
Description
0 / 350 characters
2 Configuration
Choose visibility *
Choose who can see and commit to this repository
Public
Add README
READMEs can be used as longer descriptions. About READMEs
Off
Add .gitignore
.gitignore tells git which files not to track. About ignoring files
No .gitignore
Add license
Licenses explain how others can use your code. About licenses
No license
Create repository

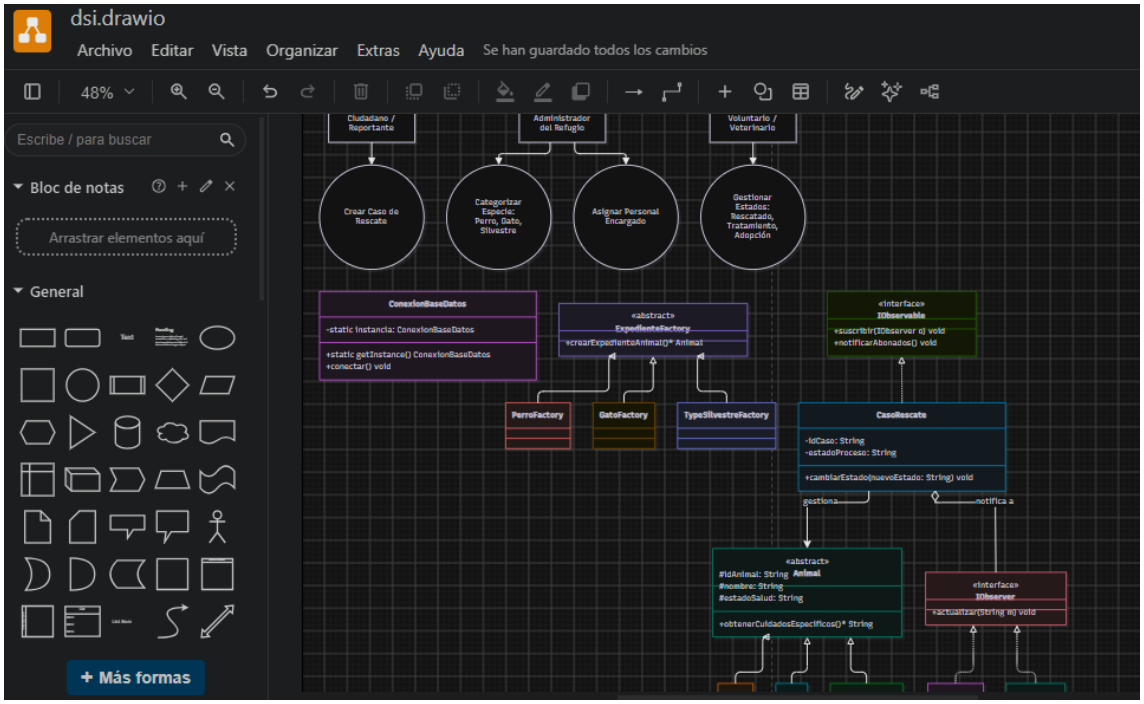

Villao@DESKTOP-R4GOQUN UCRT64 ~/OneDrive/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS GIT (master)
$ git config --global user.email "jvillao4111@utm.edu.ec"
Villao@DESKTOP-R4GOQUN UCRT64 ~/OneDrive/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS GIT (master)
$ git commit -m "commit inicial: primera version del documento"
[master (root-commit) 68a17c1] commit inicial: primera version del documento
2 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 TEST/ACT#1DSI.docx
create mode 100644 TEST/~$T#1DSI.docx
Villao@DESKTOP-R4GOQUN UCRT64 ~/OneDrive/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS GIT (master)
$


UCRT64:/c/Users/Villao/OneDrive/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS GIT
Villao@DESKTOP-R4GOQUN UCRT64 ~/OneDrive/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS GIT (master)
$ git remote add origin https://github.com/jvillao4111/S2DSI.git
git branch -M main
git push -u origin main
error: remote origin already exists.


UCRT64:/c/Users/Villao/OneDrive/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS GIT
Villao@DESKTOP-R4GOQUN UCRT64 ~/OneDrive/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS GIT (master)
$ git remote add origin https://github.com/jvillao4111/S2DSI.git
git branch -M main
git push -u origin main
error: remote origin already exists.
info: please complete authentication in your browser...
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 36.39 KiB | 18.19 MiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/jvillao4111/S2DSI.git
* [new branch] main -> main
branch 'main' set up to track 'origin/main'.


UCRT64:/c/Users/Villao/OneDrive/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS GIT
Villao@DESKTOP-R4GOQUN UCRT64 ~/OneDrive/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS GIT (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
Villao@DESKTOP-R4GOQUN UCRT64 ~/OneDrive/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS GIT (main)
$


UCRT64:/c/Users/Villao/OneDrive/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS GIT
$ # Regresar a la rama integradora
git checkout develop
Fusionar los cambios que hiciste en la feature
git merge feature/diseno-uml
Subir los cambios finales de desarrollo a GitHub
git push origin develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.
Updating 68a17c1..bb7449c
Fast-forward
TEST/ACT#1DSI.docx | Bin 41692 -> 212466 bytes
TEST/~$T#1DSI.docx | Bin 162 -> 0 bytes
2 files changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 TEST/~$T#1DSI.docx
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 188.26 KiB | 18.83 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/jvillao4111/S2DSI.git
68a17c1..bb7449c develop -> develop
Villao@DESKTOP-R4GOQUN UCRT64 ~/OneDrive/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS GIT (develop)
$



SECCIÓN 6: CONCLUSIONES

- **Primera Conclusión:** La implementación de estándares de modelado UML y patrones de diseño (Factory Method, Observer, Singleton) permitió estructurar una arquitectura lógica robusta para el sistema de rescate animal, cumpliendo de forma estricta con los principios SOLID de mantenibilidad y escalabilidad.
- **Segunda Conclusión:** El uso del flujo de trabajo profesional Gitflow mediante la consola Git Bash garantizó una correcta organización y trazabilidad histórica de las fases de análisis y diseño del software, eliminando el riesgo de pérdida de información y promoviendo buenas prácticas de ingeniería de software.

SECCIÓN 7: BIBLIOGRAFÍA

- **Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994).** *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional. *(Referencia principal para la justificación de los patrones Singleton, Factory Method y Observer utilizados en la arquitectura lógica del sistema)*.
- **Martin, R. C. (2018).** *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall. *(Sustento teórico para la aplicación de los principios SOLID de desacoplamiento y alta mantenibilidad implementados en el diseño de clases UML)*.
- **Chacon, S., & Straub, B. (2014).** *Pro Git* (2nd ed.). Apress. *(Guía metodológica para la configuración de la identidad del autor en Git Bash, el ciclo de confirmación de cambios locales mediante commits y la sincronización con repositorios remotos en GitHub)*.
- **Universidad Técnica de Manabí. (2026).** *Unidad 1: Desarrollo de software Introducción y Paradigmas de diseño - Instructivo de la Actividad #1 y Tutorial de Git y GitHub*. online.utm.edu.ec. *(Documentación institucional base utilizada para el análisis de los requerimientos funcionales del refugio de fauna y la parametrización de la topología de ramas Gitflow)*.